

PYTHON UNIT 6 QB – (4 marks)

1. Define Flask. List any two features of Flask.

1. Flask is a lightweight web framework written in Python that is used to build web applications and APIs. It provides simple tools for routing, request handling, and template rendering.
2. Flask follows a minimal design and allows developers to add extensions when needed. This makes the framework flexible and easy to customize for different projects.
3. Flask was developed by Armin Ronacher and is based on the WSGI toolkit and Jinja2 template engine. It is widely used for small and medium-sized web applications.
4. One important feature of Flask is routing support using the `@app.route()` decorator. It helps developers connect URLs with Python functions easily.
5. Another feature of Flask is template rendering using Jinja2 templates. This allows developers to create dynamic HTML pages with user data.

2. What is routing in Flask? Explain `@app.route()` with syntax.

1. Routing in Flask is the process of mapping URLs to specific Python functions in a web application. When a user enters a URL, Flask checks the route and executes the related function.
2. Flask uses the `@app.route()` decorator to create routes in the application. This decorator connects a URL path with a Python function.
3. The syntax of routing in Flask is:

```
@app.route('/path')  
def function_name():  
    return "Message"
```

This syntax tells Flask which function should run for a specific URL.

4. For example, if the route is `/hello`, Flask will execute the related function when the user visits that URL. The function then returns the response to the browser.
5. Routing helps organize web pages and makes navigation easy in Flask applications. Developers can create static as well as dynamic routes using variables in URLs.
6. Flask routing also supports different HTTP methods like GET and POST for handling requests. This makes Flask useful for forms, APIs, and database applications.

3. What is Jinja2? Why is it used in Flask?

1. Jinja2 is a template engine used in Python for generating dynamic web pages. It allows developers to combine HTML templates with data from Python programs.
2. Jinja2 is widely used in Flask because it helps create dynamic content easily. It replaces placeholders in HTML with actual values during rendering.
3. In Jinja2, variables are displayed using `{{ }}` syntax. Control statements like loops and conditions are written using `{% %}` syntax.
4. Jinja2 supports features like loops, conditions, filters, and template inheritance. These features help developers build reusable and maintainable web pages.
5. Flask uses Jinja2 to separate business logic from presentation logic. Python code handles processing, while Jinja2 manages the webpage design and display.
6. Jinja2 improves code readability and reduces duplication in web applications. It is useful for creating dynamic interfaces such as product pages, user dashboards, and forms.

4. Differentiate between GET and POST methods.

1. The GET method is used to request or retrieve data from the server. The POST method is used to send data to the server for processing.
2. In GET, the data is visible in the URL as query parameters. In POST, the data is sent inside the request body and is not shown in the URL.
3. GET requests are less secure because anyone can see the data in browser history. POST requests are more secure and are better for sensitive information like passwords.
4. GET has limited data size because the information is stored in the URL. POST can send large amounts of data without URL size limitations.
5. In Flask, GET data is accessed using `request.args`. POST data is accessed using `request.form`.
6. GET is mainly used for searching and fetching data from the server. POST is mainly used for login forms, registrations, and database updates.

5. What is a template in Flask? Give an example.

1. A template in Flask is an HTML file used to display dynamic content in web applications. Flask uses the Jinja2 template engine to render these templates.
2. Templates help developers separate webpage design from application logic. This makes the application easier to manage and maintain.
3. Flask stores HTML templates inside a special folder called templates. The `render_template()` function is used to load and display these files.
4. Example of rendering a template in Flask:

```
from flask import Flask, render_template
app = Flask(__name__)
```

```
@app.route('/')
def home():
```

```
    return render_template('index.html')

```

Templates can also display variables using Jinja2 syntax like `{{ name }}`. This allows dynamic information to appear on the webpage.

5. Flask templates support loops, conditions, and filters for creating interactive pages. They are commonly used in applications like TODO lists, student systems, and login pages.

6. What is mysql-connector in Python?

1. MySQL-connector is a Python module used to connect Python applications with MySQL databases. It helps developers perform database operations directly from Python programs.
2. This connector allows communication between Flask or Python applications and the MySQL server. Developers can store, retrieve, update, and delete data using SQL queries.
3. MySQL-connector is commonly used in CRUD-based applications. It supports operations like Create, Read, Update, and Delete in databases.
4. In Flask projects, the connector helps integrate backend applications with MySQL databases. This is useful for login systems, profile management, and student record systems.
5. The module can be installed using the command `pip install flask-mysqldb` or `mysql connector` packages. After installation, Python can access database tables and records easily.
6. MySQL-connector improves data management and makes web applications dynamic. It is widely used because it is simple, reliable, and supports structured database operations.

7. Explain the role of `app.run()` in Flask.

1. The `app.run()` function is used to start the Flask development server. It allows the Flask application to run and accept requests from users.
2. When the application starts, Flask listens for HTTP requests on the default local server address. By default, the server runs at `http://127.0.0.1:5000/`.
3. The function is usually written inside the condition:

```
if __name__ == "__main__":
```

`app.run(debug=True)` The `debug=True` option enables debug mode in Flask. It automatically reloads the server whenever code changes are made.

4. Debug mode also shows detailed error messages in the browser when problems occur. This helps developers find and fix errors easily during development.
5. The `app.run()` function is mainly used for testing and learning purposes on a local machine. It is not recommended for production-level deployment because it is a development server.

8. What is the purpose of request object in Flask?

1. The request object in Flask is used to handle data sent by the client or browser. It allows Flask applications to access user input and request information.
2. Flask uses the request object to read form data, URL parameters, cookies, and HTTP methods. It helps the server understand what the user is requesting.
3. For GET requests, data is accessed using `request.args.get()`. This method retrieves values passed through the URL query string.
4. For POST requests, data is accessed using `request.form.get()`. This retrieves values submitted through HTML forms in the request body.
5. The request object also helps developers check the type of request method being used. For example, `request.method` can identify whether the request is GET or POST.
6. The request object is important for creating dynamic and interactive web applications. It is widely used in forms, login pages, search systems, and database applications.

9. What is CRUD? Expand and define each operation.

1. CRUD is a set of basic database operations used in web applications and database management systems. CRUD stands for Create, Read, Update, and Delete.
2. Create operation is used to insert new records into the database. For example, adding a new student or user account is a create operation.
3. Read operation is used to fetch or display data from the database. It helps users view stored records such as profiles, products, or student details.
4. Update operation is used to modify existing records in the database. For example, changing a user's email address or updating marks uses the update operation.
5. Delete operation is used to remove records permanently from the database. It helps in deleting unwanted or old information from the system.
6. CRUD operations form the foundation of most database-driven applications. They are commonly used in login systems, management systems, and web applications built with Flask and MySQL.

10. What is the difference between static and dynamic web pages?

1. A static web page shows the same content to every user. A dynamic web page changes content according to user input or data from the server.
2. Static pages are created using simple HTML and CSS files. Dynamic pages use technologies like Flask, Python, databases, and Jinja2 templates.
3. In static web pages, the content does not change automatically. In dynamic web pages, the content can update based on user actions or database records.
4. Static websites are faster and easier to create because they do not require server-side processing. Dynamic websites are more interactive and support advanced features like login systems and forms.
5. Dynamic pages can display personalized information such as usernames, products, or search results. Static pages usually display fixed information like company details or simple webpages.
6. Flask is mainly used to create dynamic web applications because it supports routing, templates, forms, and database integration. Dynamic websites are more suitable for modern applications such as e-commerce, student systems, and TODO apps.

(5 – marks)

1. Explain steps to install and set up Flask environment.

1. Flask is a lightweight Python web framework used for building web applications and APIs. Before starting Flask development, Python must be installed correctly on the computer.
2. The first step is creating a virtual environment for the project. This keeps project libraries separate from other Python applications.
3. The command used to create a virtual environment is:

```
python -m venv venv
```

After creating it, the environment must be activated before installing Flask.

4. In Windows, the environment is activated using `venv\Scripts\activate`. After activation, Flask is installed using the command `pip install flask`.
5. A Python file such as `app.py` is then created for writing Flask code and routes. The application is started using `python app.py` and accessed in the browser using `http://127.0.0.1:5000/`.

2. Write a Flask program demonstrating routing for multiple URLs.

1. Flask routing is used to connect URLs with Python functions in a web application. Different routes display different responses or webpages to users.
2. Example program demonstrating multiple routes:

```
from flask import Flask
app = Flask(__name__)
```

```
@app.route('/')
def home():
    return "Homepage"
```

```
@app.route('/hello')
def hello():
    return "Hello User"
```

```
@app.route('/about')
def about():
    return "About Page"
```

```
if __name__ == "__main__":
    app.run(debug=True)
```

3. In this program, the / route displays the homepage message while /hello displays a greeting message. The /about route shows information about the application or website.
4. Flask automatically matches the entered URL with the correct function using @app.route(). The app.run(debug=True) statement starts the development server with debug mode enabled.
5. Multiple routing makes navigation easier in Flask applications and helps organize webpages properly. It is commonly used in websites containing home, login, contact, and profile pages.

3. Explain Jinja2 templating with example (variables and loops).

1. Jinja2 is a template engine used in Flask to generate dynamic HTML webpages. It combines Python data with HTML templates for displaying dynamic content.
2. Variables in Jinja2 are displayed using {{ }} syntax, while loops use {% for %} syntax. These features help in creating interactive webpages easily.
3. Example of Jinja2 template with variables and loops:

```
<h1>Welcome {{ username }}</h1>
```

```
<ul>
```

```
{% for item in items %}
```

```
<li>{{ item }}</li>
```

```
{% endfor %}
```

```
</ul>
```

4. In this example, {{ username }} displays the username passed from Flask. The loop displays all items from the list one by one dynamically.
5. Jinja2 helps separate webpage design from application logic, making code easier to manage and reuse. It is widely used in dashboards, TODO apps, shopping websites, and management systems.

4. Write a program to handle form data using GET method.

1. The GET method is used to send data through URL parameters from the browser to the server. The submitted data becomes visible in the browser URL.
2. Example Flask program using GET method:

```
from flask import Flask, request
app = Flask(__name__)

@app.route('/form', methods=['GET'])
def form():
    name = request.args.get('name')
    return f"Hello {name}"
if __name__ == "__main__":
    app.run(debug=True)
```

3. In this program, the user sends the value of name through the URL query string. Flask retrieves the value using request.args.get().
4. If the URL entered is /form?name=Meet, the output displayed will be “Hello Meet”. GET requests are commonly used for searching and retrieving data from servers.

5. Write a program to handle form data using POST method.

1. The POST method is used to send data securely from the client to the server. Unlike GET, the submitted data is not displayed in the browser URL.
2. Example Flask program using POST method:

```
from flask import Flask, request
app = Flask(__name__)

@app.route('/submit', methods=['GET', 'POST'])
def submit():
    if request.method == 'POST':
        name = request.form.get('name')
        return f"Hello {name}"

    return ""
    <form method="POST">
        <input type="text" name="name">
        <input type="submit">
    </form>
    ""

if __name__ == "__main__":
    app.run(debug=True)
```

3. In this program, the user enters a value in the HTML form and submits it using POST method. Flask receives the data using `request.form.get()`.
4. The request type is checked using `request.method == 'POST'` before processing the form data. The submitted name is then displayed in the browser as output.

6. Explain how to connect Flask with MySQL using mysql-connector.

1. Flask can be connected with MySQL to store and manage application data dynamically. The `mysql-connector` module helps establish communication between Flask and MySQL database.
2. First, MySQL Workbench and required connector packages must be installed in the virtual environment. The installation command used is `pip install flask-mysqldb`.
3. After installation, database details such as host, username, password, and database name are configured inside the Flask application. Flask then creates a connection object to communicate with MySQL.
4. Example connection code:

```
from flask import Flask
from flask_mysqldb import MySQL

app = Flask(__name__)

app.config['MYSQL_HOST'] = 'localhost'
app.config['MYSQL_USER'] = 'root'
app.config['MYSQL_PASSWORD'] = 'password'
app.config['MYSQL_DB'] = 'studentdb'

mysql = MySQL(app)
```

5. After successful connection, Flask applications can perform CRUD operations such as insert, update, delete, and fetch records. This integration is commonly used in login systems, profile management, and student databases.

7. Write steps to perform INSERT operation in MySQL using Flask.

1. INSERT operation is used to add new records into a MySQL database table from a Flask application. It is commonly used in registration forms and data entry systems.
2. First, Flask must be connected with the MySQL database using `mysql-connector` or `flask-mysqldb`. After connection, form data is collected using `request.form`.
3. Example code for INSERT operation:

```

cursor = mysql.connection.cursor()
cursor.execute(
    "INSERT INTO students(name, age) VALUES(%s, %s)",
    ("Meet", 20)
)
mysql.connection.commit()
cursor.close()

```

4. In this code, cursor.execute() runs the SQL INSERT query on the database table. The values provided inside the query are inserted as a new record.
5. The commit() function saves the changes permanently in the database. Without commit, inserted data will not be stored successfully.

8. Explain UPDATE and DELETE operations in Flask with MySQL.

1. UPDATE operation is used to modify existing records in the MySQL database. DELETE operation is used to remove records permanently from the database table.
2. In Flask, these operations are performed using SQL queries executed through a database cursor object. The application receives user input and sends it to MySQL queries.
3. Example code for UPDATE and DELETE operations:

```

# UPDATE
cursor.execute(
    "UPDATE students SET age=%s WHERE id=%s",
    (21, 1)
)

```

```

# DELETE
cursor.execute(
    "DELETE FROM students WHERE id=%s",
    (1,)
)

```

```

mysql.connection.commit()

```

4. The UPDATE query changes existing data in the table according to the given condition. The DELETE query removes the selected record permanently from the database.
5. After executing queries, commit() is used to save changes in the database successfully. These operations are important in management systems, profile editing, and CRUD applications.

9. Write a short note on project structure of Flask application.

1. Flask applications follow a simple and organized project structure for managing code, templates, and static files. A proper structure improves readability and maintenance of the application.
2. The main file of the application is usually app.py, which contains routes, functions, and Flask configuration. This file controls the overall working of the application.
3. HTML files are stored inside a special folder called templates. Flask uses Jinja2 templates from this folder to generate dynamic webpages.
4. CSS, JavaScript, and image files are generally stored inside a static folder. These files are used for webpage design and frontend functionality.
5. Flask project structures may also contain database files, configuration files, and virtual environment folders. This organized structure helps developers build scalable and reusable web applications.

10. Explain deployment of Flask application on local server.

1. Deployment means making a Flask application available so users can access it through a browser. Local deployment is mainly used for testing and development purposes.
2. The first step is creating the project structure with files such as app.py and HTML templates. Flask must also be installed using the command `pip install flask`.
3. Example code for running Flask application locally:

```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def home():
    return "Welcome to Flask"

if __name__ == "__main__":
    app.run(debug=True)
```

4. The application is executed using the command `python app.py`. Flask then starts the local development server at `http://127.0.0.1:5000/`.
5. The `debug=True` option automatically reloads the application when code changes are made and also displays detailed error messages. Local deployment is useful for testing Flask applications before cloud deployment.

(10 – marks)

1. Explain Flask architecture and request-response cycle with diagram.

1. Flask is a lightweight Python web framework based on WSGI and Jinja2 template engine. It provides components like routing, templates, and request handling for web development.
2. The main parts of Flask architecture are client, web server, Flask application, routing system, view functions, templates, and database layer. Each component helps process requests and generate responses properly.
3. The request-response cycle starts when the browser sends an HTTP request to the Flask server. The server receives the request and forwards it to the Flask application.
4. Flask checks the routing table created using `@app.route()` decorators. The matched route executes the related Python view function.
5. The view function processes business logic and may interact with databases if required. Flask then creates a response in HTML or JSON format and sends it back to the browser.
6. Jinja2 templates are used for rendering dynamic webpages with user data. This makes Flask applications interactive and user-friendly.
7. Flask architecture is simple, flexible, and suitable for APIs, CRUD systems, and student projects. The request-response cycle ensures smooth communication between client and server.
8. Diagram of Flask Architecture and Request–Response Cycle:

Client →

WSGI Server →

Flask App →

Routing →

View Function → Database/Jinja2 →

Response →

Client

2. Develop a Flask application demonstrating routing, templates, and dynamic content.

1. Flask applications use routing to connect URLs with Python functions and Jinja2 templates for dynamic webpages. Dynamic content allows webpages to display changing data from Python.
2. Flask application code:

```
from flask import Flask, render_template
```

```
app = Flask(__name__)
```

```
@app.route('/')
def home():
    return render_template("index.html")
```

```
@app.route('/user/<name>')
def user(name):
    return render_template('user.html', username=name)
```

```
if __name__ == '__main__':
    app.run(debug=True)
```

3. Example user.html template:

```
<h1>Hello {{ username }}</h1>
```

4. In this application, the / route loads the homepage template while /user/<name> accepts dynamic input from the URL. Flask sends the username value to the Jinja2 template for display.
5. Jinja2 uses {{ username }} syntax to display dynamic data inside HTML pages. This application demonstrates routing, template rendering, and dynamic content generation in Flask.

3. Design a Flask application to accept user input via form and display result using Jinja2.

1. Flask forms are used to accept user input from webpages and process it on the server. Jinja2 templates display the processed result dynamically in HTML pages.

2. Flask application code:

```
from flask import Flask, render_template, request

app = Flask(__name__)

@app.route('/')
def form():
    return render_template('form.html')

@app.route('/submit', methods=['POST'])
def submit():
    name = request.form['name']
    return render_template('result.html', name=name)

if __name__ == '__main__':
    app.run(debug=True)
```

3. Example form.html:

```
<form action="/submit" method="POST">
<input type="text" name="name">
<input type="submit">
</form>
```

4. Example result.html:

```
<h2>Hello {{ name }}</h2>
```

5. In this application, the user enters data through an HTML form and submits it using POST method. Flask processes the form data using request.form[] and passes it to the Jinja2 template for display.

4. Write a complete Flask program to perform CRUD operations using MySQL (mysql-connector).

```
from flask_mysqlldb import MySQL

app = Flask(__name__)

app.config['MYSQL_HOST'] = 'localhost'
app.config['MYSQL_USER'] = 'root'
app.config['MYSQL_PASSWORD'] = 'password'
app.config['MYSQL_DB'] = 'studentdb'

mysql = MySQL(app)

# CREATE
@app.route('/add')
def add():
    cur = mysql.connection.cursor()
    cur.execute(
        "INSERT INTO students(name) VALUES(%s)",
        ("Meet",)
    )
    mysql.connection.commit()
    return "Record Added"

# READ
@app.route('/view')
def view():
    cur = mysql.connection.cursor()
    cur.execute("SELECT * FROM students")
    data = cur.fetchall()
    return str(data)

# UPDATE
@app.route('/update')
def update():
    cur = mysql.connection.cursor()
    cur.execute(
        "UPDATE students SET name=%s WHERE id=%s",
        ("Tanvi", 1)
    )
    mysql.connection.commit()
    return "Record Updated"

# DELETE
@app.route('/delete')
def delete():
```

```

cur = mysql.connection.cursor()
cur.execute(
    "DELETE FROM students WHERE id=%s",
    (1,)
)
mysql.connection.commit()
return "Record Deleted"

if __name__ == "__main__":
    app.run(debug=True)

```

5. Develop a Student Database Viewer using Flask and MySQL with CRUD functionality.

```

from flask import Flask
from flask_mysql import MySQL

app = Flask(__name__)

app.config['MYSQL_HOST'] = 'localhost'
app.config['MYSQL_USER'] = 'root'
app.config['MYSQL_PASSWORD'] = 'password'
app.config['MYSQL_DB'] = 'college'

mysql = MySQL(app)

@app.route('/students')
def students():
    cur = mysql.connection.cursor()
    cur.execute("SELECT * FROM students")
    data = cur.fetchall()
    return str(data)

@app.route('/add')
def add():
    cur = mysql.connection.cursor()
    cur.execute(
        "INSERT INTO students(name, age) VALUES(%s,%s)",
        ("Meet", 21)
    )
    mysql.connection.commit()
    return "Student Added"

if __name__ == "__main__":
    app.run(debug=True)

```

(for 10 marks explain this program)

6. Create a TODO List Web App using Flask (add, view, delete tasks).

1. A TODO List application helps users manage daily tasks by adding, viewing, and deleting tasks. Flask is used to handle routing and task management in the application.
2. Example Flask TODO application:

```
from flask import Flask, request, redirect
```

```
app = Flask(__name__)
tasks = []
```

```
@app.route('/')
def index():
    return str(tasks)
```

```
@app.route('/add', methods=['POST'])
def add():
    task = request.form.get('task')
    if task:
        tasks.append(task)
    return redirect('/')
```

```
@app.route('/delete/<int:id>')
def delete(id):
    if 0 <= id < len(tasks):
        tasks.pop(id)
    return redirect('/')
```

```
if __name__ == '__main__':
    app.run(debug=True)
```

3. In this application, tasks are stored temporarily in a Python list. The homepage route displays all tasks currently available in the list.
4. The /add route accepts task input using POST method and adds it into the task list. The /delete/<id> route removes a selected task using its index value.
5. TODO applications demonstrate CRUD concepts and Flask routing effectively. They are simple, lightweight, and useful for learning Flask web development.

7. Explain GET vs POST methods with implementation in Flask forms.

1. GET and POST are HTTP methods used for communication between client and server in web applications. Flask supports both methods for handling form data and user requests.
2. GET method sends data through URL parameters called query strings. POST method sends data inside the HTTP request body instead of the URL.
3. GET requests are less secure because the data is visible in browser history and URLs. POST requests are more secure and suitable for sensitive information like passwords.
4. GET is mainly used for fetching or searching data from the server. POST is commonly used for login forms, registrations, and database operations.
5. Flask implementation for GET method:

```
@app.route('/form', methods=['GET'])
def form():
    name = request.args.get('name')
    return f"Hello {name}"
```

6. Flask implementation for POST method:

```
@app.route('/submit', methods=['POST'])
def submit():
    name = request.form.get('name')
    return f"Hello {name}"
```

7. In GET method, Flask accesses data using `request.args.get()` while POST uses `request.form.get()`. Choosing the correct method improves security, usability, and performance of Flask applications.

8. Discuss template inheritance in Jinja2 with example.

1. Template inheritance in Jinja2 allows developers to reuse a common layout across multiple webpages. It reduces code duplication and keeps webpage design consistent.
2. A base template usually contains common elements like header, footer, and navigation bar. Child templates extend the base template and replace selected content blocks.
3. Example of base.html:

```
<html>
<head>
<title>Website</title>
</head>
<body>
<h1>My Website</h1>

{% block content %}
{% endblock %}

</body>
</html>
```

4. Example of child template:

```
{% extends "base.html" %}

{% block content %}
<h2>Home Page</h2>
<p>Welcome User</p>
{% endblock %}
```

5. In this example, the child template inherits the structure from base.html. Only the content inside {% block content %} is replaced dynamically.
6. Template inheritance improves maintainability because changes in the base template automatically apply to all child templates. It is widely used in dashboards, shopping websites, and management systems.

9. Explain step-by-step deployment of Flask application on local server and on Heroku.

1. Deployment means making a Flask application available for users through a browser. Applications can be deployed locally for testing or on Heroku for public access.
2. For local deployment, the first step is creating the project structure containing app.py and templates folder. Flask must then be installed using `pip install flask`.
3. Example Flask application for deployment:

```
from flask import Flask
```

```
app = Flask(__name__)
```

```
@app.route('/')
```

```
def home():
```

```
    return "Flask App Running"
```

```
if __name__ == "__main__":
```

```
    app.run(debug=True)
```

4. The application is started locally using the command `python app.py`. Flask runs on `http://127.0.0.1:5000/` where the application can be tested in a browser.
5. For Heroku deployment, tools like Git, Heroku CLI, and Gunicorn are required. A Procfile is also created to define how the Flask application should run.
6. After creating the Heroku app, project files are pushed using Git commands. Heroku then hosts the Flask application on the internet for global access.
7. Local deployment is mainly used for development and testing while Heroku deployment is used for public hosting. Deployment helps convert Flask projects into real-world web applications accessible by users.

10. Design a mini web application integrating routing, templates, forms, and database connectivity.

1. A mini Flask web application combines routing, forms, templates, and database connectivity to create dynamic websites. Such applications are commonly used in login systems and student management projects.
2. Example Flask application:

```
from flask import Flask, request, render_template
from flask_mysqlldb import MySQL
```

```
app = Flask(__name__)
```

```
app.config['MYSQL_HOST'] = 'localhost'
app.config['MYSQL_USER'] = 'root'
app.config['MYSQL_PASSWORD'] = 'password'
app.config['MYSQL_DB'] = 'college'
```

```
mysql = MySQL(app)
```

```
@app.route('/')
def form():
    return render_template('form.html')
```

```
@app.route('/submit', methods=['POST'])
def submit():
    name = request.form['name']

    cur = mysql.connection.cursor()
    cur.execute(
        "INSERT INTO students(name) VALUES(%s)",
        (name,)
    )
    mysql.connection.commit()
```

```
    return f"{name} Added Successfully"
```

```
if __name__ == "__main__":
    app.run(debug=True)
```

3. Example form.html:

```
<form action="/submit" method="POST">
<input type="text" name="name">
<input type="submit">
</form>
```